

Univerzitet u Banjoj Luci
Prirodno matematički fakultet

Boris Petković(121/22)

Prevođenje Kotlin koda u UML dijagram sekvenci

(seminarski rad iz predmeta „Konstrukcija kompilatora“)

Predmetni profesor:Milana Grbić

Asistent:Milan Predojević

Banja Luka 2026.

Sadržaj:

1.Uvod.....	3
2. Teorijska osnova	4
3. Organizacija projekta.....	5
4. Parsiranje i implementacija	6
5. Primjeri	8
6.Zaključak	13
7.Literatura	14
8.Dodatak.....	14

1.Uvod

UML (Unified Modeling Language) dijagrami predstavljaju alat za vizuelno modeliranje i dokumentovanje arhitekture sistema. Oni omogućavaju razvojnim timovima da na objektivan način sagledaju strukturu koda, definišu odnose između komponenti i jasno identifikuju ključne attribute i metode unutar klasa.

Ipak, tradicionalni pristup ručne izrade dijagrama često se suočava sa značajnim preprekama. Proces je dugotrajan i podložan ljudskim greškama, a možda i najznačajniji problem leži u brznoj desinhronizaciji. Kako se izvorni kod neprestano mijenja i evoluira, statični dijagrami koji se ne ažuriraju automatski ubrzo prestaju da dobro oslikaju stvarno stanje sistema, postajući tako netačni i neprecizni. Iz tih razloga, raste potreba za razvojem alata koji omogućavaju automatizaciju procesa dokumentovanja, odnosno direktno generisanje UML modela iz samog izvornog koda.

Glavni cilj ovog seminarskog rada je konstrukcija prevodioca koji automatizuje ovaj proces za programski jezik Kotlin. Fokus je na razvoju alata koji analizira Kotlin izvorni kod i iz njega automatski generiše UML dijagram sekvenci, sa praktičnom primjenom u softverskom inženjerstvu, tako što pruža bržu dokumentaciju..Metodološki, rješenje se oslanja na nekoliko ključnih tehnologija:

Realizacija projekta obuhvata nekoliko faza prevođenja:

- **Leksička i sintaksna analiza:** Korišćenjem alata ANTLR, izvorni Kotlin kod se transformiše u apstraktno sintaksno stablo (AST) koje predstavlja gramatičku strukturu programa.
- **Semantička analiza:** Implementirana u jeziku Python, ova komponenta obilazi stablo (koristeći *Visitor* obrazac) kako bi identifikovala objekte, redoslijed njihovog kreiranja i međusobne pozive metoda.
- **Generisanje dijagrama:** Prikupljeni podaci se prevode u PlantUML format, koji finalno generiše vizuelni prikaz sekvenci.

2. Teorijska osnova

Kotlin je moderan, statički tipiziran programski jezik koji se izvršava na Java virtuelnoj mašini (JVM). Razvijen od strane kompanije JetBrains, Kotlin je dizajniran da bude potpuno kompaktabiln sa Javom, ali uz znatno sažetiju sintaksu i veću bezbjednost koda. Njegova filozofija se oslanja na jasnoću i produktivnost, što ga čini idealnim za objektno-orijentisano modeliranje. U kontekstu ovog rada, Kotlin je pogodan jer njegova struktura poziva metoda i kreiranja objekata nudi direktnu analogiju sa elementima UML dijagrama sekvenci.

Dijagram sekvenci prikazuje interakciju između objekata u hronološkom redoslijedu. Ovi dijagrami fokusirani su na tok poruka kroz vrijeme.

Linije života (Lifelines) predstavljaju individualne učesnike u sistemu. U našem prevodiocu, svaki instancirani objekat u Kotlinu dobija svoju liniju života. Na primjer, deklaracija `val pump = WaterPump()` u kodu rezultuje kreiranjem linije života za objekat `pump`.

Poruke i pozivi metoda se prikazuju kao horizontalne strelice. Kada u Kotlinu napišemo `controller.startIrrigation(50)`, to se prevodi u sinhronu poruku usmjerenu od pozivaoca prema objektu `controller`. UML standard nalaže da se naziv metode i parametri ispišu iznad strelice.

Povratne vrijednosti (Return Messages) signaliziraju kraj operacije i povratak kontrole. Naš alat prepoznaje eksplicitne povratke, poput `return flowRate`, ili dodjele rezultata varijablama kao što je `val result = math.calculateSum(10, 20)`. U UML-u se ovi povratni tokovi prikazuju isprekidanom strelicom koja nosi naziv vraćene vrijednosti (npr. `--> result`).

Kontrolne strukture se u UML-u prikazuju kroz kombinovane fragmente.

- **Alternativa (alt):** Mapira se na Kotlinov if-else blok. Na primjer, provjera `if (currentDay == 2)` kreira alt blok na dijagramu, razdvajajući tokove za ispunjen i neispunjen uslov.
- **Petlja (loop):** Mapira se na while ili for petlje. Konstrukcija `while (day <= 3)` rezultuje loop fragmentom koji uokviruje sve interakcije unutar tijela petlje.

Aktivacioni blokovi su vertikalni pravougaonici koji označavaju trajanje izvršavanja operacije na liniji života. Naš generator automatski ispisuje komande `activate` i `deactivate` kako bi vizuelno naglasio koji je objekat trenutno zadužen za procesiranje poruke.

Na ovaj način, analizom apstraktnog sintaksnog stabla (AST) dobijenog iz Kotlina, svaki programski obrazac dobija svoj grafički ekvivalent. Automatizacijom ovog procesa dobijamo alat koji preslikava logiku izvršavanja koda u standardizovanu UML notaciju.

3. Organizacija projekta

Arhitektura ovog sistema dizajnirana je tako da omogući transformaciju statičkog izvornog koda u dinamički prikaz interakcija. Projekat je organizovan modularno, čime je postignuta jasna podjela između procesa parsiranja, ekstrakcije logike i same vizuelizacije.

Struktura projekta oslanja se na sljedeću organizaciju foldera i datoteka:

- grammars/: Sadrži datoteke KotlinLexer.g4 i KotlinParser.g4. One definišu formalnu gramatiku jezika Kotlin, na osnovu koje ANTLR generiše komponente za parsiranje.
- gen/: U ovom folderu se nalaze automatski generisane Python datoteke (KotlinLexer, KotlinParser, KotlinParserVisitor i KotlinParserListener). One služe kao interfejs za rad sa apstraktnim sintaksnim stablom (AST).
- src/: Centralni dio logike sistema smješten je u tri ključne datoteke:
 1. main.py: Upravljačka skripta koja koordinira cijeli proces.
 2. uml_visitor.py: Sadrži klasu KotlinSemanticAnalyzer koja vrši semantičku analizu i prikuplja podatke o interakcijama.
 3. plantuml_generator.py: Prevodi prikupljene podatke u tekstualni format razumljiv PlantUML alatu.
- input/ i output/: Folderi namijenjeni za smještaj ulaznih .kt datoteka i rezultujućih .puml i .png dijagrama.
- plantuml.jar: Eksterni alat koji vrši konačno iscrtavanje dijagrama na osnovu generisanog koda.

Proces počinje u main.py, gdje se ulazni fajl učitava i proslijeđuje leksičkom analizatoru. Rezultat je niz tokena koji parser organizuje u apstraktno sintakšno stablo (AST).

Nakon toga, ključnu ulogu preuzima uml_visitor.py. Ovaj vizitor se fokusira na tok izvršavanja (sequence_flow). On obilazi funkcije (poput main), prati redoslijed kreiranja objekata i, što je najvažnije, detektuje pozive metoda. Specifičnost ove implementacije je mehanizam "inline-ovanja" (_inline_method_call), koji omogućava vizitoru da "uđe" u tijelo pozvane metode i zabilježi dublje nivoe interakcije, čime se simulira stek poziva (call stack).

Prikupljeni podaci (objekti, redoslijed kreiranja, pozivi, dubina ugniježđenosti i povratne vrijednosti) se organizuju u strukturirani rječnik pomoću metode `get_sequence_data()`. Ovi podaci se zatim proslijeđuju u `plantuml_generator.py`.

Generator koristi logiku stakova (`call_stack` i `block_stack`) kako bi ispravno ispisao PlantUML sintaksu. On vodi računa o tome kada treba aktivirati (`activate`) ili deaktivirati (`deactivate`) liniju života objekta, te kako uokviriti interakcije unutar kontrolnih struktura kao što su `alt` za uslovna grananja ili `loop` za petlje.

Finalni tekstualni kod se čuva u `.puml` fajlu, koji `main.py` šalje PlantUML-u na iscrtavanje grafičkog dijagrama u `.png` formatu.

Ukratko, arhitektura projekta definisana je kroz šest faza:

1. Definicija gramatike – Putem ANTLR `.g4` datoteka.
2. Leksička i sintaksna analiza – Generisanje AST stabla iz Kotlin koda.
3. Upravljanje procesom – Preko `main.py` skripte.
4. Semantička ekstrakcija – `uml_visitor.py` izvlači hronološki tok interakcija i dubinu poziva.
5. Transformacija koda – `plantuml_generator.py` kreira tekstualni opis dijagrama sekvenci.
6. Vizuelizacija – PlantUML alat generiše konačnu sliku interakcija.

4. Parsiranje i implementacija

Prva faza je leksička analiza, koju obavlja leksički analizator generisan pomoću ANTLR alata na osnovu datoteke `KotlinLexer.g4`. On razbija izvorni tekst na osnovne gradivne jedinice — tokene. Na primjer, ključne riječi poput `class`, `fun`, `if` ili `while`, identifikatori objekata i operatori dodjele (`=`) prepoznaju se kao zasebni entiteti.

Nakon toga slijedi sintaksna analiza kojom upravlja `KotlinParser.g4`. Parser kombinuje tokene u hijerarhijsku strukturu poznatu kao apstraktno sintaksno stablo (AST). U ovom stablu, pozivi

metoda su predstavljeni kroz čvorove poput `functionCall`, dok su kontrolne strukture definisane pravilima `ifStatement` ili `whileStatement`.

Klasa `KotlinSemanticAnalyzer` smještena u datoteci `uml_visitor.py` nasljeđuje `KotlinParserVisitor` i koristi *Visitor* obrazac za selektivni obilazak čvorova AST-a.

Semantička analiza za dijagrame sekvenci fokusira se na redoslijed događaja. Ključne operacije uključuju:

- **Praćenje objekata:** Vizitor bilježi svako instanciranje klase (npr. `val pump = WaterPump()`) i dodjeljuje mu jedinstvenu liniju života.
- **Mehanizam inline-ovanja:** Funkcija `_inline_method_call` omogućava vizitoru da, pri nailasku na poziv metode, privremeno "skoči" u tijelo te metode u drugoj klasi. Ovo je ključno za prikazivanje dubokih interakcija između objekata koje čine suštinu dijagrama sekvenci.
- **Identifikacija povratnih vrijednosti:** Sistem analizira `return` naredbe i dodjele varijabli (npr. `val flow = ...`) kako bi identifikovao nazive i vrijednosti koje se vraćaju pozivaocu.

Prevođenje Kotlinovih kontrolnih struktura u UML kombinovane fragmente je implementirano kroz metode `visitIfStatement` i `visitWhileStatement` ubacuju specifične markere (`alt_start`, `else_start`, `loop_start`) u listu događaja. Ovi markeri omogućavaju generatoru da u finalnom dijagramu uokviri određene interakcije, prikazujući logička grananja i ponavljanja u kodu.

Posljednja faza implementirana je u klasi `PlantUMLSequenceGenerator` unutar datoteke `plantuml_generator.py`. Ovaj modul uzima strukturirane podatke iz vizitora i prevodi ih u tekstualni PlantUML format.

Generator koristi mehanizam stack-ova za upravljanje vizuelnim elementima:

1. **call_stack:** Prati aktivne pozive kako bi ispravno ispisao `activate` i `deactivate` komande, čime se formiraju vertikalni blokovi aktivnosti na dijagramu.
2. **block_stack:** Osigurava da se svi pozivi započeti unutar jednog `if` ili `while` bloka pravilno zatvore isprekidanom povratnom strelicom prije nego što se blok završi ili pređe u `else` granu.

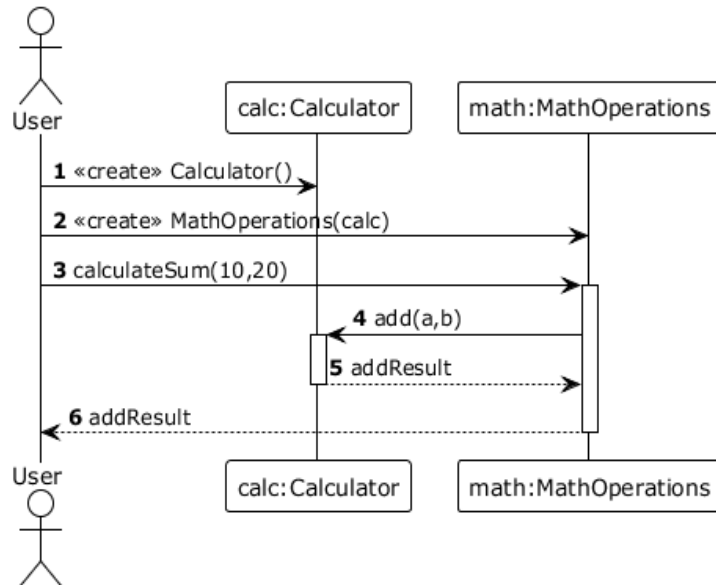
Konačni rezultat je `.puml` datoteka koja, nakon procesiranja kroz `plantuml.jar`, postaje grafički prikaz koji hronološki i logički oslikava izvršavanje Kotlin programa.

5. Primjeri

Simple2.kt

Ovaj primjer demonstrira osnovne interakcije između dva objekta i mehanizam prosljeđivanja povratnih vrijednosti kroz različite slojeve poziva metoda. Fokus je na linearnom toku izvršavanja i pravilnom prikazivanju stack-a poziva u UML notaciji.

```
fun main() {  
    val calc = Calculator()  
    val math = MathOperations(calc)  
    val result = math.calculateSum(10, 20)  
    println("Rezultat je: $result")  
}  
  
class MathOperations(val calc: Calculator) {  
    fun calculateSum(a: Int, b: Int): Int {  
        println("Računam zbir $a i $b")  
        return calc.add(a, b)  
    }  
}  
  
class Calculator {  
    fun add(x: Int, y: Int): Int {  
        println("Sabiranje: $x + $y")  
        val sum = x + y  
        return sum  
    }  
}
```

- Akter User (koji predstavlja main funkciju) vrši instanciranje objekata calc i math. U UML-u su ovi koraci označeni porukom <<create>> koja vodi direktno do linija života objekata. Poziv metode calculateSum(10, 20) aktivira objekat O1 (math). Unutar te aktivacije, O1 upućuje novi poziv add(a, b) prema objektu O0 (calc), što otvara novi, ugniježđeni aktivacioni blok. Nakon što Calculator izvrši sabiranje, rezultat (addResult) se vraća isprekidanom strelicom nazad do objekta math. Na kraju, isti rezultat se prosljeđuje nazad do korisnika, čime se završava životni ciklus ove interakcije.

Complex1.kt

- Ovaj primjer predstavlja složeniji model logike pametne kuće koji upravlja grijanjem i hlađenjem prostorije. Primjer je dizajniran da pokaže kako prevodilac rukuje kontrolnim strukturama kao što su petlje i uslovna grananja, te kako se ugniježđeni pozivi preslikavaju na kompleksne interakcije između više lifelina.

```

fun main() {
    val heater = Heater()

    val thermostat = Thermostat(heater)

    val room = Room(thermostat, 18)

    val temperatures = listOf(18, 22, 26, 19, 24)

    for (temp in temperatures) {

```

```

        println("\n=== Provjera za temperaturu: $temp°C ===")

        room.currentTemp = temp

        room.checkTemperature()

    }
}

```

```

class Room(val thermostat: Thermostat, var currentTemp: Int) {

    fun checkTemperature() {

        println("Trenutna temperatura je $currentTemp°C")

        if (currentTemp < 20) {

            println("Hladno je, palim grijanje")

            thermostat.setHeating(true)

        } else {

            println("Prevrucé je, palim hlađenje")

            thermostat.setCooling(true)

        }

    }

}

```

```

class Thermostat(val heater: Heater) {

    fun setHeating(enable: Boolean) {

        if (enable) {

            println("Termostat: uključujem grijanje")

            heater.turnOn()

        } else {

            println("Termostat: isključujem grijanje")

            heater.turnOff()

        }

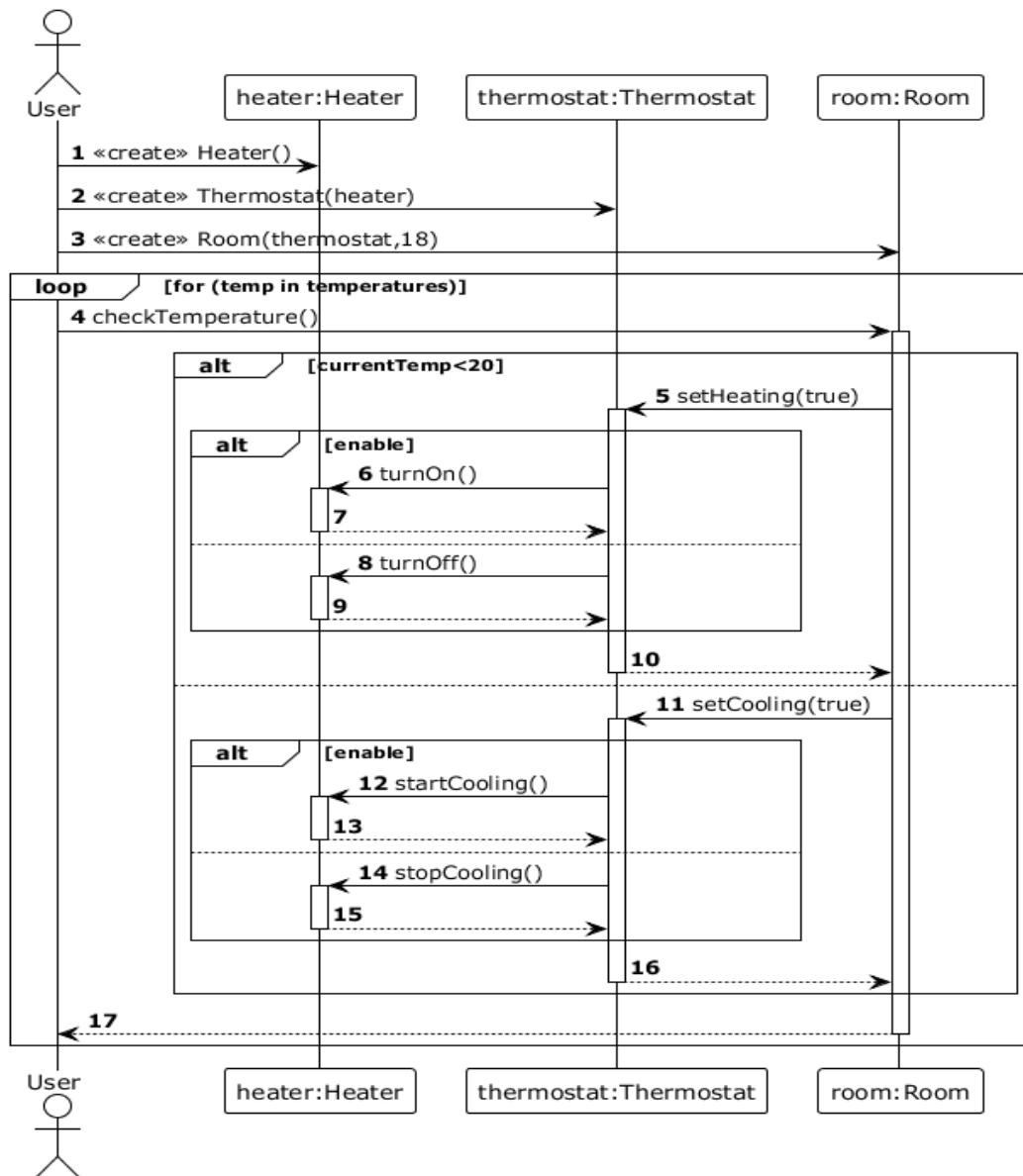
    }

}

```

```
fun setCooling(enable: Boolean) {  
    if (enable) {  
        println("Termostat: uključujem hlađenje")  
        heater.startCooling()  
    } else {  
        println("Termostat: isključujem hlađenje")  
        heater.stopCooling()  
    }  
}  
}
```

```
class Heater {  
    fun turnOn() {  
        println("Grijalica radi, grije prostoriju")  
    }  
  
    fun turnOff() {  
        println("Grijalica je ugašena")  
    }  
  
    fun startCooling() {  
        println("Klima radi, hladi prostoriju")  
    }  
  
    fun stopCooling() {  
        println("Klima je ugašena")  
    }  
}
```



Centralni dio dijagrama zauzima loop blok koji mapira Kotlinovu for petlju. Unutar ovog bloka, sistem iterira kroz listu temperatura, vršeći provjeru za svaku vrijednost pojedinačno.

Prvi alt blok unutar metode checkTemperature odlučuje da li će se aktivirati grijanje ili hlađenje na osnovu uslova `currentTemp < 20`.

Unutar metoda `setHeating` i `setCooling` nalaze se dodatni ugniježđeni alt blokovi koji provjeravaju status `enable`. Ovi blokovi upravljaju direktnim pozivima hardverskih funkcija grijalice (`turnOn`, `turnOff`, itd.).

Dijagram prikazuje dubinu poziva. Vidljivo je kako soba (`Room`) upravlja termostatom (`Thermostat`), koji dalje komanduje samoj grijalici (`Heater`). Svaki nivo poziva otvara novi aktivacioni blok, a isprekidane strelice na kraju svakog bloka signaliziraju povratak kontrole na viši nivo nakon obavljene akcije.

6. Zaključak

Glavni cilj ovog rada bio je razvoj softverskog alata koji automatizuje proces prevođenja izvornog Kotlin koda u vizuelne UML dijagrame sekvenci. Realizovani prevodilac uspješno identifikuje ključne elemente dinamičkog ponašanja sistema, uključujući instanciranje objekata, ugniježdene pozive metoda i kontrolne strukture poput petlji i uslovnih grananja.

Završni rezultat ovog projekta jeste predstavljanje grafičkih dijagrama, ali je i dalje sistem koji je izgrađen na principima klasične konstrukcije kompilatora. Kroz faze leksičke, sintaksne i semantičke analize, sirovi tekst koda se transformiše u strukturirane podatke koji omogućavaju preciznu vizualizaciju toka izvršavanja programa.

Dobijeni rezultati potvrđuju da ovakav alat značajno olakšava razumijevanje arhitekture sistema i omogućava brzu izradu dokumentacije. Projekat je dokazao praktičnu primjenljivost teorijskih koncepata prevođenja u svakodnevnom razvoju softvera, pružajući objektivnu osnovu za analizu i dalji razvoj kompleksnih softverskih rješenja.

7.Literatura

- JetBrains: Kotlin Programming Language Documentation.
<https://kotlinlang.org/docs/home.html>
- PlantUML Official Documentation : <https://plantuml.com>
- Kotlin Specification (release gramatika) : <https://github.com/Kotlin/kotlin-spec/tree/release/grammar>

8.Dodatak

Kompletan kod projekta, zajedno sa svim gramatičkim fajlovima, skriptama i primjerima, dostupan je na GitHub repozitorijumu:

<https://github.com/BorisPetkovic70/TranslatingKotlinToUMLSequenceDiagram>